

Contexto compartilhado entre agentes

24 de agosto de 2026 · estado **implementado** (Fase 2) · para outro dev

Irmãos de um run Auto compartilham a sessão Claude **sem pagar a API para copiar o transcript**. Handoff e o first prompt injetam um **ponteiro** para chunks em disco. Duas CLIs diferentes ainda não compartilham o mesmo composer — compartilham o **conteúdo** da sessão, não o processo.

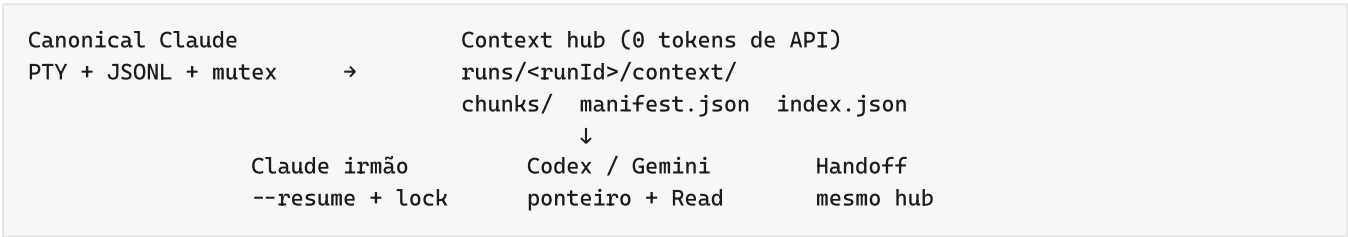
Por que isso existe

No Auto, vários panes (Claude, Codex, Gemini...) trabalham no mesmo pedido. Antes, cada pane nascia com uma conversa nova e o Flashwork tentava passar contexto colando histórico no `initialInput` — até ~48 KB. Isso cobrava tokens só para copiar o passado, duplicava sessões Claude (`already claimed` → `fresh writer`) e não dividia o trabalho.

A Fase 2 troca o dump por um **hub local** e **uma** sessão Claude canônica por run.

Arquitetura

Um writer Claude (PTY + JSONL + `sessionId`) → o Flashwork indexa esse JSONL em disco, sem modelo → irmãos Claude fazem `--resume` da mesma id; Codex/Gemini/etc. leem chunks.



Disco: `{profile}/runs/{runId}/context/` (mesmo root do journal). Chunks extrativos (~4 KB, agrupados por arquivo tocado). Índice lexical (termos + paths). Sem embedding API.

1. Como compartilhar entre sessões?

Uma sessão Claude canônica por run Auto: um JSONL, um writer PTY, um `sessionId` (`PromptRun.canonicalClaudeSessionId`).

DE → PARA	O QUE ACONTECE
Claude → Claude	Não cria conversa nova. <code>--resume</code> da id canônica. Se outro pane segura o writer, espera o lock (até 120 s).
Claude → Codex / Gemini / OpenCode	Não entram no processo do Claude. Consomem o hub em <code>runs/<id>/context/</code> .
Handoff Claude ↔ Codex	<code>materialize_agent_handoff</code> grava o mesmo hub em <code>handoffs/<id>/context/</code> . <code>contextPath</code> = <code>manifest.json</code> .

O que não é compartilhado como processo vivo: duas CLIs não digitam no mesmo TUI. Dois processos Claude não escrevem a mesma `sessionId` ao mesmo tempo — por isso o mutex.

2. Dá para reusar sessão sem gastar tokens?

Para copiar histórico para outro modelo: sim — 0 tokens de API. Parse do JSONL, split em chunks (`CHUNK_CHAR_CAP = 4000`) e índice invertido são locais. Sem embed, sem chat/completions para montar o hub.

O próximo turno útil ainda cobra o que aquele CLI enviar (slice + chunks que ele Read). A economia é não reingirir ~48 KB como first prompt.

AÇÃO	TOKENS DE MODELO
Parse JSONL + chunk + índice local	0
Bootstrap do irmão (ponteiro, $\leq$ 2k chars)	irrisório, um shot
Agente lê o chunk <code>0003.md</code>	só aquele chunk
Segundo Claude <code>--resume</code> a mesma sessão	0 para duplicar (não há duplicata)
Capsule antigo (~48k no <code>initialInput</code>)	proibido

Refresh do watcher (`shouldIngestContext` + debounce 800 ms): 0 chamadas de chat.

3. O que acontece com as sessões Claude? Duplica?

Não, dentro de um run.

- Primeira lane Claude: `--session-id` com UUID uma vez → `rememberCanonicalClaude` + persist no `PromptRun`.
- Claude posterior no mesmo run: `--resume` dessa id, ou espera o lock.
- Conflito de claim dessa id **não** cai no `already claimed; starting a fresh writer`.
- `planAutoLanes` emite no máximo um worker `agent  $\equiv$  'claude'`.
- Retry de early-exit na sessão canônica não força UUID novo.

Outros CLIs nunca ganham uma sessão Claude forjada. Ganham arquivos. O lock de writer é in-memory e é restaurado do `PromptRun` persistido no `setRun` / hydrate.

4. Como o contexto passa? Ainda injeta no first prompt?

Só um ponteiro. Auto (`buildRunBootstrapInput`): papel + slice + `contextDir` absoluto + “leia `manifest.json` e só os chunks que batem nos seus arquivos.” Para um prompt curto, o texto fica < 2k chars.

Handoff (`buildIndexBootstrap`): o mesmo ponteiro + o pedido do usuário. Não o corpo da capsule. O modelo puxa chunks com Read quando precisa. Qualidade é extrativa (turnos reais + arquivos tocados), não um resumo feito por LLM.

```
[Flashwork Auto] You are a worker (Codex) for run {id}.
Do the assigned slice. Do not paste or request the sibling transcript.
Shared context is on disk at "{abs}/context". Read manifest.json and only
the chunks that match your files. The index was built locally with no API cost.
```

```
{slice do usuário}
```

Antes x agora

PERGUNTA	ANTES	AGORA
Compartilhar	Board + first prompt gordo; sessão nova por pane	Uma sessão Claude + hub local
Reuso sem tokens	Resume só no mesmo pane; Auto mintava chat novo	Resume canônico; índice de graça para construir
Irmãos Claude	Segundo pane = JSONL novo (<code>fresh writer</code>)	Mesmo JSONL, lock, <code>--resume</code>
Passar contexto	Transcript/capsule no <code>initialInput</code>	Caminho; chunks em disco

Limites honestos

`prepare_agent_handoff` ainda monta um rascunho interno da conversa para ter texto a chunkar. O que mudou é o que entra no `initialInput`.

Se `prepare/materialize` falha (ou a origem não é Claude/Codex), o fallback ainda aponta o journal + o pedido do usuário.

O próximo turno de qualquer CLI continua cobrado pelo provedor. Duas CLIs não compartilham um composer vivo — restrição dos produtos, não um TODO nosso.

Onde olhar no código

PEÇA	ARQUIVO
Mint / resume da sessão canônica	<code>src/lib/promptRun/startPromptRun.ts</code>
Mutex de writer	<code>src/lib/promptRun/claudeWriterLock.ts</code>
<code>--session-id</code> vs <code>--resume</code>	<code>src/lib/sessionLaunch.ts</code> , <code>useXtermSession.ts</code>
Bootstrap ponteiro	<code>src/lib/promptRun/bootstrapPrompt.ts</code>
Handoff ponteiro	<code>src/lib/promptRun/executeHandoff.ts</code>
Chunk + índice (TS)	<code>src/lib/promptRun/contextChunks.ts</code>
Hub + ingest (Rust)	<code>src-tauri/src/context_hub.rs</code>
Ingest ao vivo	<code>src/hooks/usePromptRunWatcher.ts</code>

Flashwork Orchestrator · briefing interno para compartilhar · não é spec de produto. Testes no momento da implementação: 451 passing (Vitest) + testes do módulo `context_hub`.